

Credit Risk Decisioning Engine

An End-to-End, Explainable, Cost-Sensitive Credit Risk System

Full Project Plan & Guide

1. Problem Statement

1.1 Business Context

Consumer lenders — banks, credit unions, and fintech lenders — approve or deny millions of loan applications every year. Every decision carries two opposing risks:

- **Approving a borrower who defaults** → the lender loses the principal (minus any recovery), a loss that can be 10–20x larger than the profit from a single good loan.
- **Denying a borrower who would have repaid** → the lender loses the interest income and the customer relationship, and — if systematic — creates fairness/regulatory exposure.

Traditional credit scoring (FICO-style scorecards) is transparent and regulator-approved but linear, coarse, and slow to adapt. Modern ML models (XGBoost, LightGBM) are more accurate but are frequently rejected internally or by regulators because they are hard to explain, poorly calibrated, and prone to silently degrading as the borrower population shifts over time (a phenomenon called **population drift**).

This creates a real, unsolved-in-most-portfolios problem: **how do you get the accuracy of modern ML without losing the explainability, calibration, and monitoring discipline that regulated lending requires?**

1.2 The Problem, Precisely

Given a borrower's application-time financial profile (income, credit utilization, delinquency history, loan purpose, employment, etc.), predict the probability that the loan will default (charge-off) rather than be paid in full — and convert that probability into an actual lending **decision**: approve/deny, and at what interest rate/risk tier — in a way that is:

1. **Accurate** — outperforms a naive "approve everyone" or simple-rules baseline.
2. **Calibrated** — a predicted 12% default probability should mean ~12% of similar borrowers actually default, not just "riskier than 11%."
3. **Cost-aware** — decisions are optimized against real dollar costs of false negatives (missed defaulters) vs. false positives (wrongly denied good borrowers), not abstract accuracy.

4. **Explainable** — every denial can be traced to specific, human-readable reasons (a legal requirement under the U.S. Equal Credit Opportunity Act / Regulation B — lenders must supply "adverse action" reasons).
5. **Fair** — does not produce systematically worse outcomes for one age/income/geographic group without justification.
6. **Monitorable** — detects when the borrower population has drifted enough that the model needs retraining, before losses accumulate.

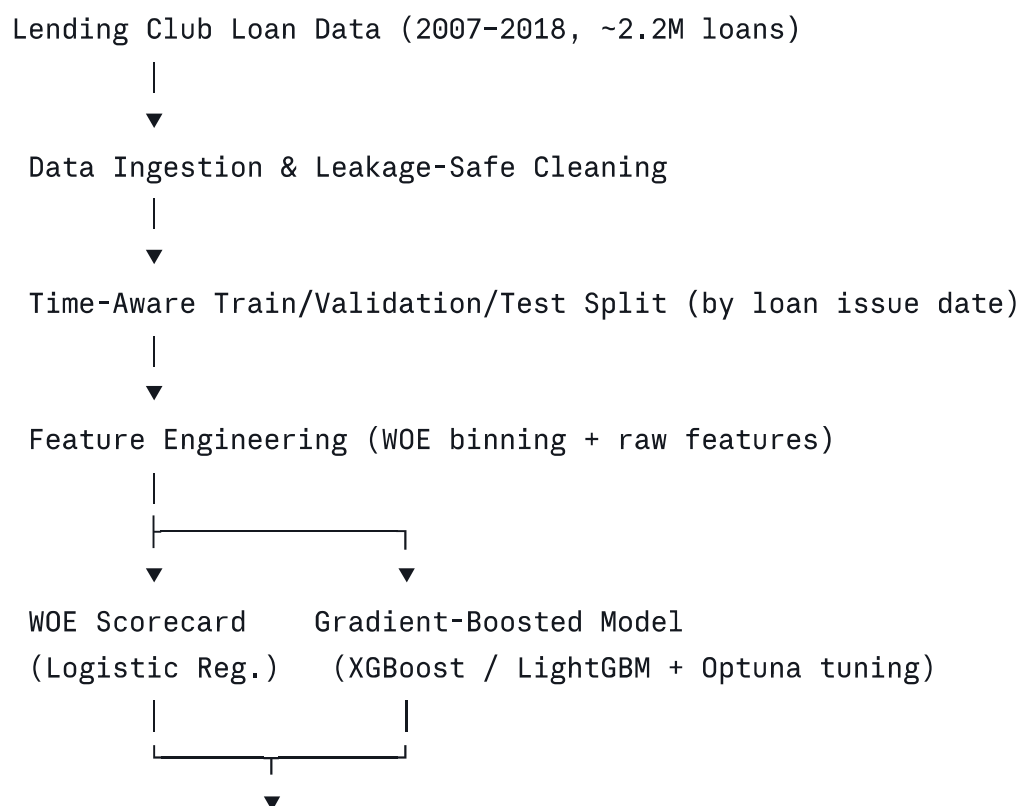
1.3 Why This Is Not "Just Another Kaggle Notebook"

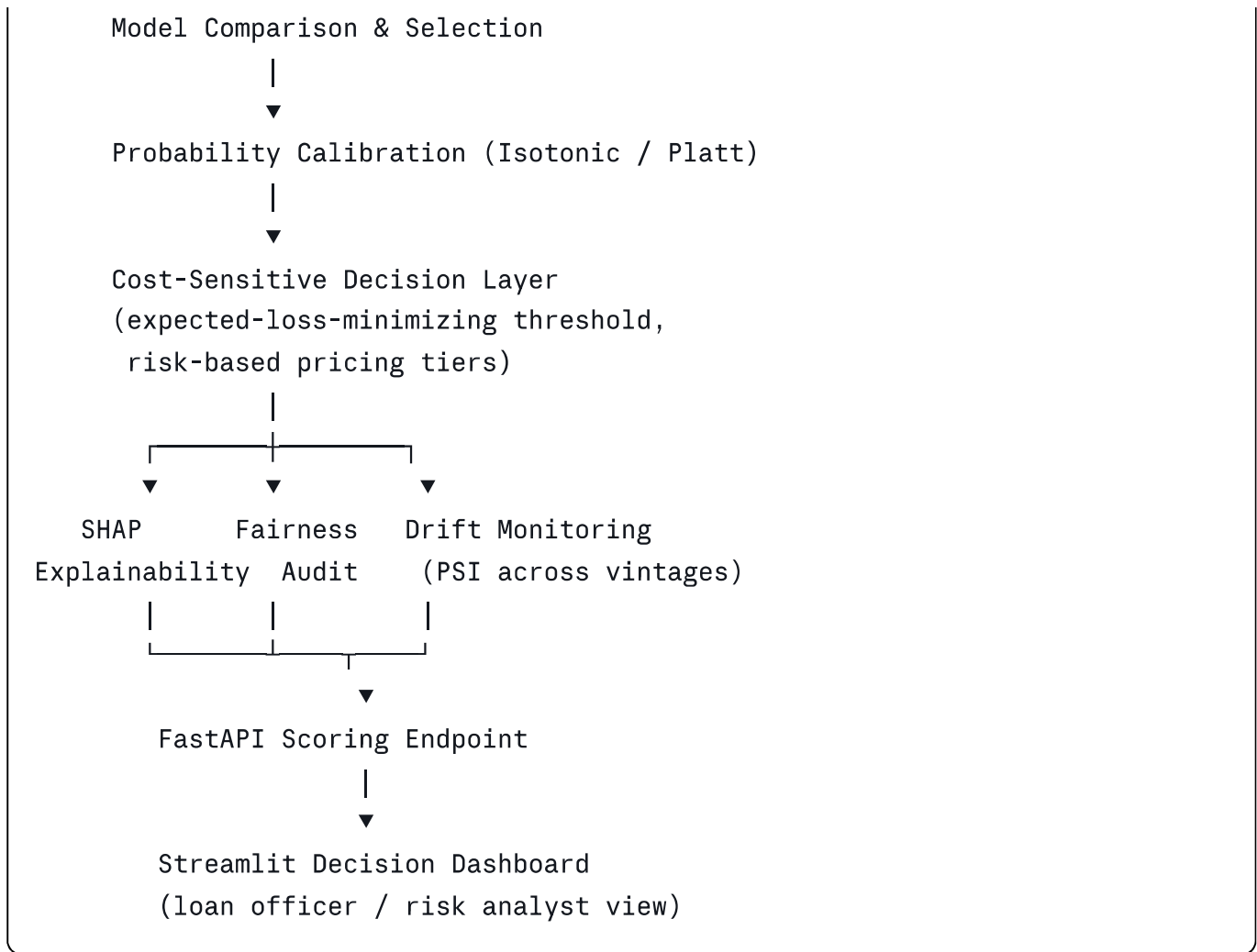
Almost every public "loan default prediction" project stops at step 1 (accuracy) and reports an AUC score. That's a classroom exercise. A real credit risk system is judged on steps 2–6 — this project builds all six, which is what turns "I trained an XGBoost model" into "I understand how credit risk models actually get approved, deployed, and monitored inside a lender."

2. Proposed Solution — System Overview

One-line pitch: *A decisioning engine that doesn't just predict default risk — it turns that prediction into a calibrated, explainable, cost-optimal lending decision, and watches itself for drift over time.*

2.1 High-Level Architecture





2.2 Why Lending Club Data

- Free, public, ~2.2M loans (2007–2018) with real `issue_d` (issue date) — enables genuine temporal validation and drift analysis, which a static Kaggle snapshot (like "Give Me Some Credit") cannot support.
- Includes the lender's **own assigned grade and interest rate** — giving you a real benchmark: *"does my model's risk ranking agree with or improve on Lending Club's actual pricing?"*
- Rich feature set: FICO range, DTI, revolving utilization, employment length, home ownership, loan purpose, delinquency counts, public records, verification status.
- Widely used in actual credit risk research, so your project reads as "practitioner-grade dataset," not an intro competition.

2.3 What Makes Each Layer Legitimate (not decorative)

Layer	Why it's there, not just "cool tech"
WOE Scorecard	This is the actual industry-standard, regulator-approved approach — including it (and comparing to it) proves you understand what banks really deploy, not just what Kaggle rewards.
Time-aware split	Random splits leak future information into training in credit risk (economic cycles matter). This is the single most common mistake in public notebooks — fixing it is a real signal of rigor.
Calibration	An 80% AUC model can still produce garbage probabilities. Lenders need probabilities, not just rankings, because pricing and loss provisioning depend on the actual number.
Cost-sensitive threshold	Converts "risk score" into "decision" — the actual deliverable a lender needs, not just a number nobody acts on.
SHAP + reason codes	Directly maps to a real legal requirement (ECOA/Reg B adverse action notices) — this is the detail that proves finance-domain understanding, not generic "explainability is nice" thinking.
Fairness audit	The other side of the same legal requirement (disparate impact under fair lending laws) — pairs naturally with reason codes.
PSI drift monitoring	Models decay. A model that predicted well in 2015 may be dangerously wrong by 2018 as the borrower population and macro conditions shift. Detecting this before losses accumulate is core MLE/production thinking.
FastAPI endpoint	Shows you can operationalize a model, not just leave it in a notebook.

3. Detailed Phase-by-Phase Plan

Phase 0 — Setup (Day 0-1)

- Create repo with clean structure (see §5)
- Download Lending Club dataset (Kaggle: "Lending Club Loan Data" or the accepted/rejected loan data variant)
- Set up environment (`requirements.txt` / `environment.yml`), MLflow tracking initialized from day one

- Write initial README skeleton with the business problem stated up front

Phase 1 — Data Understanding & Leakage-Safe Cleaning (Days 2-5)

- Inventory all 140+ raw columns; classify each as: **available at application time** (usable) vs. **post-origination / outcome-derived** (must drop — e.g. `total_pymnt`, `recoveries`, `last_pymnt_amnt`, `out_prncp`)
- This classification itself becomes a documented artifact — "identified and excluded 40+ leakage-risk features" is a genuinely strong, concrete resume line
- Define target: `loan_status` → binary (`Charged Off`/`Default` = 1, `Fully Paid` = 0); drop `Current`/in-progress loans with justification
- Handle missing values with documented, defensible logic (not blanket dropna)
- Document class imbalance (Lending Club default rate is typically ~15-20%)

Phase 2 — Time-Aware Split & EDA (Days 5-8)

- Split by `issue_d`; e.g. train on 2007-2015, validate on 2016, test (out-of-time) on 2017-2018
- EDA framed around business questions, not generic plots:
 - How does default rate vary by loan grade, purpose, DTI band?
 - How did default rates shift across economic periods (2008 crisis cohort vs. 2016 cohort)?
 - What does the FICO/interest rate relationship look like vs. actual outcomes?
- Deliverable: EDA notebook + a 1-page "key findings" summary (not just plots — findings)

Phase 3 — Feature Engineering (Days 8-11)

- WOE (Weight of Evidence) binning for scorecard model, with IV (Information Value) ranking to justify feature selection
- Engineered features: DTI bands, credit utilization categories, delinquency aggregate score, income-adjusted loan amount, employment stability proxy
- Explicit feature documentation: what it means, why it's predictive, whether it's a fairness-sensitive proxy

Phase 4 — Modeling (Days 11-16)

- **Baseline:** WOE + Logistic Regression scorecard (the "what a bank actually uses" baseline)

- **Challenger:** XGBoost/LightGBM with Optuna-driven hyperparameter search, 5-fold stratified CV within the training window (never across the time split)
- Track every run in MLflow: params, AUC, KS-statistic, Gini, PR-AUC
- Compare models on **KS-statistic and Gini** (the actual metrics credit risk teams report), not just AUC
- Select final model with documented rationale (interpretability vs. lift tradeoff explicitly discussed)

Phase 5 — Calibration (Days 16–18)

- Reliability diagrams (predicted probability vs. observed default rate) for both models
- Apply isotonic regression / Platt scaling where needed
- Report Brier score before/after calibration

Phase 6 — Cost-Sensitive Decision Layer (Days 18–21)

- Define transparent cost assumptions (documented, cited to industry loss-given-default literature — not invented)
 - Cost of False Negative $\approx$ loan principal $\times$ (1 – recovery rate)
 - Cost of False Positive $\approx$ foregone interest income over expected loan life
- Derive the expected-loss-minimizing decision threshold (not 0.5 by default — show why)
- Build risk-tier pricing: bucket predicted PD into 5–7 grades, assign risk-based interest rates, compare against Lending Club's actual assigned grade/rate — quantify agreement/improvement
- Deliverable: a short "credit policy memo" — this is the artifact that makes it look like a real business deliverable, not a modeling exercise

Phase 7 — Explainability & Fairness (Days 21–25)

- SHAP global feature importance + local (per-loan) explanations
- Auto-generate adverse action reason codes: top-N SHAP-negative features per denied applicant, mapped to plain-English reasons (e.g. "high revolving credit utilization," "limited credit history")
- Fairness audit: disparate impact ratio (80% rule) across age proxy, income band, geography (state); document findings honestly, including any concerning results and how they'd be mitigated
- Deliverable: fairness report as its own document — this is the single highest-signal artifact for finance-domain seriousness

Phase 8 — Drift Monitoring (Days 25-27)

- Compute PSI (Population Stability Index) for key features and for the model's output score, across quarterly vintages in the out-of-time test period
- Visualize drift over time; define/simulate an alert threshold ("PSI > 0.25 triggers retraining review")
- This demonstrates the MLE-flavored "what happens after deployment" thinking most DS portfolios skip entirely

Phase 9 — Serving & Packaging (Days 27-30+)

- FastAPI `/score` endpoint: takes applicant features, returns PD, calibrated probability, risk tier, decision, top reason codes
- Thin Streamlit dashboard: loan officer view — enter/upload applicant, see decision + explanation + drift status
- Final README rewrite: lead with business problem, decision impact, and a clear findings summary — not "here's what I ran"

4. Evaluation Metrics (What You'll Actually Report)

Category	Metrics
Discrimination	AUC-ROC, Gini coefficient, KS-statistic, PR-AUC
Calibration	Brier score, reliability diagram, Expected Calibration Error
Business impact	Expected loss reduction (\$) vs. baseline policy, approval rate at chosen threshold, risk-tier pricing agreement with Lending Club grades
Fairness	Disparate impact ratio (80% rule) across protected-adjacent segments
Stability	PSI over time (feature-level and score-level)

5. Repository Structure

```
credit-risk-decisioning-engine/  
|  
└─ README.md
```

```
├─ requirements.txt
├─ config/
│   └─ config.yaml          # paths, cost assumptions, thresholds
│
├─ data/
│   ├── raw/                # gitignored
│   └─ processed/
│
├─ notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_feature_engineering.ipynb
│   └─ 03_model_comparison.ipynb
│
├─ src/
│   ├── data/
│   │   ├── load_data.py
│   │   ├── clean.py
│   │   └─ leakage_audit.py
│   ├── features/
│   │   ├── woe_binning.py
│   │   └─ feature_engineering.py
│   ├── models/
│   │   ├── train_scorecard.py
│   │   ├── train_xgboost.py
│   │   ├── calibrate.py
│   │   └─ evaluate.py
│   ├── decisioning/
│   │   ├── cost_matrix.py
│   │   ├── threshold_optimization.py
│   │   └─ risk_pricing.py
│   ├── explainability/
│   │   ├── shap_explainer.py
│   │   └─ reason_codes.py
│   ├── fairness/
│   │   └─ disparate_impact.py
│   └─ monitoring/
│       └─ psi_drift.py
│
├─ reports/
│   ├── eda_findings.md
│   ├── credit_policy_memo.md
│   ├── fairness_report.md
│   └─ drift_report.md
│
```

```
├─ app/
|   ├── api.py                # FastAPI
|   └─ dashboard.py          # Streamlit
|
├─ artifacts/                # saved models, calibrators
├─ mlruns/                   # MLflow tracking
└─ tests/
    ├── test_leakage_audit.py
    ├── test_woe_binning.py
    └─ test_decisioning.py
```

6. Deliverables Checklist

- ☐ Leakage audit document (which features excluded, and why)
 - ☐ EDA findings summary (business-question-framed, not generic plots)
 - ☐ WOE scorecard model + IV table
 - ☐ Tuned XGBoost/LightGBM model with MLflow-tracked experiments
 - ☐ Model comparison report (KS, Gini, AUC, PR-AUC — scorecard vs. ML)
 - ☐ Calibration report (reliability diagrams, Brier score)
 - ☐ Credit policy memo (cost assumptions, optimal threshold, expected loss impact)
 - ☐ Risk-tier pricing comparison vs. Lending Club's actual grades
 - ☐ SHAP explainability report + sample adverse-action letters
 - ☐ Fairness/disparate-impact report
 - ☐ PSI drift report across vintages
 - ☐ FastAPI scoring endpoint
 - ☐ Streamlit decision dashboard
 - ☐ Final polished README with business narrative up front
-

7. Resume / Portfolio Framing

Project title: *Credit Risk Decisioning Engine — Explainable, Cost-Sensitive Loan Default Prediction System*

Resume bullet (example):

Built an end-to-end credit risk decisioning system on 2.2M Lending Club loans (2007–2018): WOE scorecard benchmarked against a calibrated, Optuna-tuned XGBoost model; derived a cost-sensitive decision threshold from expected-loss minimization; generated SHAP-based adverse-action explanations and audited fairness across

borrower segments; monitored population drift across loan vintages using PSI. Served via FastAPI with a Streamlit decision dashboard.

Skills this demonstrates (mapped to what you're targeting):

- **Data Scientist:** statistical rigor (WOE/IV, calibration, KS-statistic), hypothesis-driven EDA, fairness analysis
 - **ML Engineer:** model tuning (Optuna), experiment tracking (MLflow), model serving (FastAPI), drift monitoring — the "post-deployment" thinking most portfolios skip
 - **AI Engineer** (secondary, since your other projects cover this heavily): shows you can be equally rigorous outside of LLM/agent systems, broadening your range
-

8. Immediate Next Steps

1. Download the Lending Club dataset from Kaggle and confirm exact column availability
2. Run the leakage audit pass (Phase 1) before writing a single line of modeling code
3. Set up the repo skeleton and MLflow tracking
4. Lock the time-based train/val/test split boundaries

I can start scaffolding the repo structure and the data ingestion/leakage-audit script whenever you're ready.